ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE

Computer Vision Walmart (CVW) — Detección y Monitoreo de Quiebres de Stock en Góndolas mediante Detección de Objetos

· Portafolio de Título (Capstone) · Ingeniería en Informática · Duoc UC

Grupo	Grupo 4
Integrantes	David Esteban Salvatierra Cayupi, David Yáñez, Francisco Ignacio Fernández, Francisco Javier Merino Jofré
Cliente / contraparte	Walmart Chile
Artefactos de referencia	Notebook Proyecto_CVW_modelo_v1.ipynb (Google Colab) · pesos best.pt · demo web en GitHub Pages · Informe Final Proyecto APT (Fase 2)

Ficha del documento

Fecha	Revisión	Autor	Modificación
25/09/2026	3.0	Grupo 4	Alineación con el notebook del modelo (detección YOLOv8n, procesamiento por lotes, semáforo, métricas reales) e incorporación de la estructura IEEE 830 (Parte III).
06/10/2026	4.0	Grupo 4	Alineación con el Informe Final del Proyecto APT (Fase 2): umbrales del semáforo (Verde 100–41 %, Amarillo 40–21 %, Rojo 20–0 %); la base de datos SQLite se reemplaza por una carpeta de ubicaciones dentro del proyecto; las alertas pasan de Telegram Bot a correo electrónico y se incorpora la API; objetivos, áreas y competencias actualizados; se agregan las secciones 11.2 y 12; se eliminan notas de borrador y resaltados.

2. Descripción Proyecto APT

Nombre del proyecto	**Computer Vision Walmart (CVW) — Detección y Monitoreo de Quiebres de Stock en Góndolas mediante Detección de Objetos**
Área (s) de desempeño(s)	Inteligencia Artificial y Deep Learning, Visión por Computador (Computer Vision), Desarrollo y Arquitectura de Software, y Modelamiento de Bases de Datos.
Competencias	• Desarrollar soluciones de software utilizando técnicas que permitan sistematizar el desarrollo y mantenimiento (despliegue modular en VS Code y Antigravity con Python e inferencia de YOLOv8n). • Construir modelos de datos para soportar los requerimientos de la organización de acuerdo a un diseño escalable. • Gestionar proyectos informáticos ofreciendo alternativas para la toma de decisiones bajo marcos de trabajo ágiles (Scrum estructurado en 5 Sprints y control de Carta Gantt). • Realizar pruebas de validación con buenas prácticas de la industria.

3. Fundamentación Proyecto APT

Relevancia del proyecto APT	• La falta de disponibilidad de productos en góndola es una de las mayores fuentes de pérdida de ingresos en el comercio masivo, muchas veces porque el producto está físicamente en bodega pero no se encuentra repuesto en sala. Los quiebres de stock no detectados a tiempo deterioran además la experiencia del cliente. • El proyecto se sitúa en las salas de venta y pasillos de supermercados (adaptado al entorno de Walmart Chile): un entorno altamente dinámico, con tránsito continuo de clientes, variación constante de la iluminación ambiental (reflejos, sombras), ángulos visuales inclinados y productos apilados que cambian de disposición continuamente. • Impacta al personal de reposición y logística de sala (optimiza sus tiempos al evitar rondas visuales a ciegas y dirigirse directamente al pasillo crítico), a la

	administración y supervisión de tienda (reduce las ventas perdidas por quiebres no detectados a tiempo y mejora el cumplimiento del planograma) y a los clientes (encuentran los productos exhibidos y disponibles para su compra). ● Aporte de valor: automatizar la inspección visual de góndolas mediante el procesamiento por lotes de imágenes, determinando cuándo una bandeja cae a un nivel crítico de desabastecimiento (20 % o menos de stock visible) y disparando alertas con su ubicación física exacta. Se logra sin incurrir en hardware costoso (básculas o sensores IoT en estantes) y sin comprometer la ciberseguridad corporativa, ya que opera desacoplado de las cámaras en directo.
Descripción del Proyecto APT	● El proyecto consiste en una solución de visión por computador que procesa lotes de imágenes pregrabadas de góndolas de supermercado, mediante un modelo YOLOv8 Nano de detección de objetos (bounding boxes). En lugar de reconocer miles de SKU, el modelo distingue solo dos clases: 'product' (producto visible) y 'missing' (hueco o espacio vacío). Con los conteos de ambas clases se calcula el porcentaje de stock visible de la bandeja y se clasifica en un semáforo operativo (Verde 100–41 %, Amarillo 40–21 %, Rojo 20–0 %). Cuando el resultado es Rojo, el sistema obtiene la ubicación (Pasillo / Góndola / Nivel) desde una carpeta dentro del propio proyecto, que reemplaza a la base de datos externa, y envía una alerta automática por correo electrónico al personal de reposición. Adjuntar la imagen anotada como evidencia es una funcionalidad deseable, no obligatoria.
Pertinencia del proyecto con el perfil de egreso	● Aplica competencias clave de la carrera: análisis y desarrollo de modelos de datos, gestión de proyectos informáticos, programación de software, entrenamiento e inferencia de redes neuronales y estructuración modular del código en un IDE profesional (VS Code) para su despliegue.
Relación con los intereses profesionales	● Permite profundizar en Inteligencia Artificial aplicada, Visión por Computador, Machine Learning y Arquitectura de Software, contribuyendo a un perfil profesional enfocado en la automatización de procesos e IA operacional, áreas de alta demanda en la industria actual.
Factibilidad de	

desarrollo del Proyecto APT	• El desarrollo del proyecto es factible dentro del semestre académico mediante una planificación ágil (Scrum) que combina las horas de la asignatura con trabajo autónomo. Esto permite sostener reuniones de avance con el docente y, cuando sea posible, con el cliente, presentando iteraciones constantes que mejoran el producto en cada sprint. • Los recursos y materiales: • Hardware: laptop de gama media como estación de trabajo y ambiente de inferencia; entrenamiento en Google Colab con GPU NVIDIA Tesla T4, sin costos fijos de infraestructura. • Software y herramientas: Python, Visual Studio Code, Antigravity, Ultralytics YOLOv8, Albumentations, OpenCV, Roboflow (SDK), servicio de correo electrónico para las alertas y Google Colab. • Datos: dataset público de Roboflow Universe 'empty-spaces-in-a-supermarket-hanger' (versión 3), etiquetado con bounding boxes en dos clases (missing / product), utilizado para el entrenamiento y la calibración inicial del modelo. • Factores facilitadores: comunicación y acompañamiento técnico periódico con el cliente, asegurando un levantamiento claro de los requerimientos operacionales y la validación de las alertas. • Disponibilidad de modelos pre-entrenados y librerías robustas de visión por computador. • Estrategia técnica de 'void detection': detectar huecos y espacios vacíos en lugar de clasificar miles de productos individuales, lo que reduce drásticamente el volumen de etiquetado necesario. • Conocimiento previo adquirido en asignaturas de programación, bases de datos, inteligencia artificial y aprendizaje automático. • Factores obstaculizadores: • Ruido visual y distorsiones en las imágenes: sombras, variaciones lumínicas, desenfoque o ángulos de captura pueden afectar la precisión del modelo (mitigado con data augmentation). • Falta de datos reales etiquetados de salas Walmart y posibles retrasos en la entrega de material por parte del cliente; sin acceso a CCTV/RTSP ni a sensores IoT en estantes. Dataset reducido de imágenes por limitaciones de infraestructura, lo que obligó a aplicar técnicas de aumentación. En síntesis, el proyecto es factible: se apoya en áreas que el equipo domina (aprendizaje de máquinas, bases de datos e integración de software) y ya cuenta con un modelo entrenado y validado (ver sección 11). La duración acotada del semestre se

	aborda con sprints e incrementos funcionales.

A. PARTE II

4. Objetivos

Objetivo general	• Desarrollar un sistema automatizado de monitoreo e identificación de quiebres de stock en góndolas de supermercado que, mediante visión por computador con YOLOv8n, procese lotes de imágenes, calcule el porcentaje de stock visible por góndola y lo clasifique según el criterio designado para el recinto en que se implementará.
Objetivos específicos	• Adquirir, estructurar y particionar un dataset de imágenes de góndolas etiquetado con cajas delimitadoras (bounding boxes) en dos clases (missing y product), aumentando el set de entrenamiento mediante la librería Albumentations con transformaciones lumínicas y geométricas. • Entrenar y optimizar una red neuronal YOLOv8 Nano en Google Colab con aceleración por GPU Tesla T4, evaluando su rendimiento mediante métricas estándar de la industria (Precisión, Recall, mAP50 y mAP50-95). • Codificar la lógica algorítmica del semáforo operacional (Verde 100–41 %, Amarillo 40–21 %, Rojo 20–0 %) e integrar el servicio de despacho de alertas automáticas.

5. Metodología

Descripción de la Metodología
Se implementó una Metodología Ágil basada en el marco Scrum, organizada en 5 Sprints iterativos de 2 a 3 semanas de duración a lo largo de las 18 semanas de la asignatura. Esto permitió realizar entregas incrementales del sistema, validar el rendimiento de la IA de manera continua y ajustar la solución rápidamente según los resultados obtenidos. Ciclo de Sprints:

- Sprint 1 - Adquisición y Preparación de Datos: descarga del dataset de góndolas desde Roboflow Universe (formato YOLOv8), partición limpia de validación (15 %, semilla 42) antes de aumentar, y aumento del set de entrenamiento con Albumentations (3 copias por imagen).

- Sprint 2 - Entrenamiento y Prototipado del Modelo IA: entrenamiento del modelo YOLOv8 Nano (detección con bounding boxes) en Google Colab con GPU Tesla T4 (100 épocas, batch 16, tamaño 640 px; parada anticipada en la época 91, con el mejor modelo en la época 71). Evaluación de Precisión, Recall y mAP por clase para distinguir productos presentes vs. espacios vacíos.

- Sprint 3 - Base de Datos y Semáforo Operativo: desarrollo del script que calcula el porcentaje de stock visible y lo clasifica según las reglas del semáforo (Verde 100–41 %, Amarillo 40–21 %, Rojo 20–0 %). Se integró al proyecto una carpeta que reemplaza a la base de datos externa y permite asociar cada detección con su ubicación (ej.: Pasillo 1, Góndola 3, Nivel 2).

- Sprint 4 - Modularización en VS Code y Servicio de Alertas: traslado del código del notebook de Colab a una estructura modular de backend en VS Code, integración del sistema de notificación por correo electrónico ante estado Rojo y creación de la API.

- Sprint 5 - Pruebas de Integración y Validación de Latencia: pruebas del flujo completo (Lote de imágenes -> Inferencia YOLOv8 -> Cálculo del semáforo -> Consulta de ubicación en la carpeta del proyecto -> Alerta por correo electrónico) con lotes de imágenes que simulan estanterías reales, verificación de la latencia de inferencia y ajuste de falsos positivos generados por sombras u oclusiones.

Pertinencia: Scrum fue indispensable por la naturaleza empírica de los proyectos de Inteligencia Artificial. Permitió desacoplar las pruebas de entrenamiento intensivo en la nube del desarrollo del software de integración local, facilitando la toma de decisiones rápidas (como priorizar la detección por cajas delimitadoras frente a polígonos complejos) sin retrasar los entregables de ingeniería.

6. Evidencias

Tipo de evidencia (avance o final)	Nombre de la evidencia	Descripción	Justificación
Avance	Dataset y Modelo	Dataset de góndolas (Roboflow v3)	Demuestra la viabilidad

	Base de Detección (YOLOv8n)	particionado y aumentado, y primer modelo YOLOv8 Nano entrenado (best.pt) con métricas de desempeño: mAP50 72,8 %, Precisión 78,5 %.	técnica de la detección visual de huecos mediante bounding boxes.
Avance	Estructura de ubicaciones y Semáforo Operativo	Carpeta de ubicaciones dentro del proyecto (pasillos / góndolas / niveles), que reemplaza a la base de datos externa, y script en Python que calcula el % de stock visible y lo clasifica en Verde, Amarillo o Rojo (20–0 %).	Valida la lógica de negocio y la asociación de datos antes de la integración total.
Final	Demostración Funcional e Informe Final	Demostración donde el sistema procesa un lote de imágenes de góndolas, calcula el semáforo de cada bandeja y dispara la alerta por correo electrónico con la ubicación cuando el estado es Rojo. Incluye la imagen anotada que entrega el modelo YOLO tras su ejecución.	Certifica el cumplimiento de los objetivos propuestos para la Asignatura Capstone.

7. Plan de Trabajo

Plan de Trabajo Proyecto APT

Competencia o unidades de competencias	Nombre de Actividades/Tareas	Descripción Actividades/Tareas	Recursos	Duración de la actividad	Responsable[1]	Observaciones
análisis y desarrollo de modelos de datos	Sprint 1: Preparación del Dataset	Descargar el dataset de Roboflow, validar las imágenes de góndolas, generar el split train / valid y aplicar data augmentation.	Roboflow Universe (dataset v3), Google Colab, Python, Albumentations.	1-2 semana	David Yañez Francisco Fernandez, David Salvatierra, Francisco Merino.	Asegurar que las imágenes tengan diversidad de ángulos y luces.
gestión de proyectos informáticos	Sprint 2: Entrenamiento YOLO	Entrenar el modelo YOLOv8 Nano (detección) en Google Colab y medir Precisión, Recall y mAP por clase.	Python, Ultralytics, Google Colab (GPU T4).	3 semana	Francisco Fernandez / David Yañez	Requerirá varias iteraciones de ajuste de hiperparámetros.
análisis y desarrollo de modelos de datos /gestión de proyectos	Sprint 3: Semáforo y carpeta de ubicaciones	Organizar la carpeta de ubicaciones (pasillos / góndolas /	Python, carpeta de ubicaciones dentro del proyecto.	3 - 4 semana	Francisco Fernandez / David Yañez	Mantener la fórmula de cálculo ligera para no ralentizar el

1.

informáticos		niveles) dentro del proyecto y programar el cálculo del % de stock visible con los umbrales del semáforo.				procesador.
arquitectura de software	Sprint 4: Modularización en VS Code y Alertas	Estructurar el proyecto en VS Code en módulos limpios, crear la API e integrar el envío de alertas automáticas por correo electrónico.	VS Code, servicio de correo electrónico, API del sistema.	3-4 semana	David Salvatierra / Francisco Merino.	Facilitador: uso de librerías estándar de Python.
calidad de software	Sprint 5: Pruebas de Integración y Validación de Latencia	Realizar pruebas completas con lotes de imágenes de góndolas con distintos niveles de stock, verificar la latencia de inferencia y corregir posibles falsas alarmas.	Lotes de imágenes de prueba, entorno VS Code.	6 semana	David Salvatierra / Francisco Merino.	Analizar el comportamiento ante sombras y oclusiones por personas pasando.

8. Carta Gantt

Planificación de 18 semanas organizada en 3 fases, sincronizada con los 5 sprints de la metodología.

Actividad	Fase 1						Fase 2											Fase 3
	S 1	S 2	S 3	S 4	S 5	S 6	S 7	S 8	S 9	S 10	S 11	S 12	S 13	S 14	S 15	S 16	S 17	S 18
Identificar el problema																		
Levantamiento de Requerimientos																		
Elaboración del Informe de los requerimientos (ERS)																		
Com																		

paracciones / optimizaciones de los diferentes datasets de prueba																			
Entrenamiento del modelo YOLOv8n (detección)																			
Lógica del semáforo: cálculo del % de stock visible y umbral crític																			

o (≤ 20 %)																			
Detección de espacios vacíos (clase missing) en las góndolas																			
Detección de productos presentes (clase product)																			
Integración completa del modelo con la carpeta de ubicacio																			

nes y las alertas																		
Pruebas finales: errores y falsos positivos																		
Cierre del proceso																		
Redacción de la documentación de los pasos seguidos y de la implementación																		
Conclusiones																		

PARTE III

9. Introducción

9.1 Propósito	Este documento especifica los requisitos funcionales y no funcionales del sistema Computer Vision Walmart (CVW) y documenta el pipeline de IA implementado en el notebook Proyecto_CVW_modelo_v1. Está dirigido al equipo de desarrollo (Grupo 4), al docente guía de Capstone y a la contraparte de Walmart Chile, y sirve como base de acuerdo para la validación del producto. Esta versión (4.0) queda alineada con el Informe Final del Proyecto APT (Fase 2).
9.2 Ámbito del sistema	**Incluye:** procesamiento por lotes de imágenes pregrabadas de góndolas; detección de productos y espacios vacíos con YOLOv8 Nano; cálculo del % de stock visible; semáforo operativo; consulta de ubicación en la carpeta de ubicaciones del proyecto; alertas automáticas por correo electrónico; API del sistema; demo web de visualización. **No incluye:** conexión a cámaras en vivo (CCTV/RTSP); sensores IoT o básculas en estantes; base de datos externa; reconocimiento de SKU o marcas individuales; reposición automática ni integración con el sistema de inventario de Walmart.
9.3 Definiciones, acrónimos y abreviaturas	• **OOS (Out-of-Stock):** quiebre de stock; producto ausente en la góndola. • **Void detection:** enfoque que detecta huecos en lugar de clasificar cada SKU. • **YOLO (You Only Look Once):** familia de modelos de detección de objetos en una sola pasada. • **Bounding box (bbox):** rectángulo que delimita un objeto detectado (formato YOLO: clase, x, y, ancho, alto normalizados). • **Precisión (P):** proporción de detecciones que son correctas. • **Recall (R):** proporción de objetos reales que el modelo logra detectar. • **mAP50 / mAP50-95:** precisión media promedio con umbral de IoU 0,50 / promedio entre 0,50 y 0,95. • **IoU:** intersección sobre unión entre la caja predicha y la real. • **NMS:** Non-Maximum Suppression; elimina detecciones duplicadas. • **Data augmentation:** generación de variantes sintéticas de las imágenes para robustecer el modelo. • **Época / Early stopping:** una pasada completa por el dataset / detención anticipada al no mejorar. • **best.pt:** archivo de pesos del modelo con mejor desempeño en validación. • **SKU:** unidad de mantenimiento de stock (producto individual). • **RTSP / CCTV:** protocolo de streaming / circuito cerrado de televisión. • **RF / RNF:** requisito funcional / no funcional.
9.4 Referencias	• IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.

	• Kruchten, P. (1995). The 4+1 View Model of Architecture. IEEE Software, 12(6). • Documentación de Ultralytics YOLOv8 (docs.ultralytics.com). • Documentación de Albumentations (albumentations.ai). • Roboflow Universe: workspace 'final-project-object-detection-for-instore-inventory-management', proyecto 'empty-spaces-in-a-supermarket-hanger-1upsp', versión 3. • Informe Final Proyecto APT, Fase 2 (Grupo 4, Duoc UC). • Notebook del equipo: Proyecto_CVW_modelo_v1.ipynb (Google Colab).
9.5 Visión general del documento	La Parte I y II describen el proyecto APT (fundamentación, objetivos, metodología y planificación). La Parte III sigue la estructura de IEEE 830: introducción (9), descripción general (10), documentación técnica del pipeline de IA (11), lógica de negocio del semáforo (12), requisitos específicos (13) y resumen de arquitectura según el modelo 4+1 (14).

10. Descripción General

10.1 Perspectiva del producto	CVW es un sistema independiente que opera por lotes (batch), sin conexión a la infraestructura de cámaras de Walmart. Se compone de: (1) módulo de ingesta de lotes de imágenes; (2) módulo de inferencia YOLOv8n; (3) módulo de cálculo del semáforo; (4) carpeta de ubicaciones dentro del proyecto, que reemplaza a la base de datos externa; (5) módulo de alertas por correo electrónico; (6) API del sistema; y (7) demo web de visualización en GitHub Pages. El modelo se entrena fuera de línea en Google Colab y se exporta como best.pt.
10.2 Funciones del producto	• Preparar el dataset: descarga, partición train/valid y data augmentation. • Entrenar y validar el modelo de detección de dos clases (missing / product). • Procesar lotes de imágenes y detectar productos y huecos por bandeja. • Calcular el % de stock visible y clasificar cada bandeja en Verde, Amarillo o Rojo. • Asociar cada imagen a su ubicación física (Pasillo / Góndola / Nivel). • Enviar alertas automáticas por correo electrónico, con la ubicación, ante estado Rojo. • Visualizar los resultados en la demo web.

10.3 Características de los usuarios

Tipo de usuario	Formación / conocimientos	Interacción con el sistema
Reponedor	Manejo básico de correo electrónico.	Recibe por correo electrónico las alertas de estado Rojo con la ubicación, y ejecuta la reposición.
Supervisor / Jefe de sala	Conocimientos de operación de sala y ofimática básica.	Revisa el semáforo de las góndolas y prioriza rondas de reposición.

Tipo de usuario	Formación / conocimientos	Interacción con el sistema
Administrador técnico (equipo de desarrollo)	Python, YOLO, estructura de carpetas del proyecto.	Carga los lotes de imágenes, mantiene la carpeta de ubicaciones y reentrena o actualiza el modelo.

10.4 Restricciones	• Sin acceso a streaming RTSP/CCTV por políticas de ciberseguridad corporativa: solo imágenes pregrabadas por lotes. • Sin hardware adicional (básculas o sensores IoT) por costo y problemas de descalibración. • Clasificación limitada a dos clases (nc = 2): 0 = missing, 1 = product; no se identifican SKU. • Modelo de detección con bounding boxes (YOLOv8 Nano); se descartó la segmentación de polígonos y SAM por simplicidad y velocidad. • Entrenamiento sujeto a los recursos y límites de sesión de Google Colab (GPU Tesla T4). • El dataset de entrenamiento es público (Roboflow Universe) y no proviene de salas de Walmart. • Sin base de datos externa: la información de ubicación de las góndolas se guarda en una carpeta dentro del propio proyecto.
10.5 Suposiciones y dependencias	• Las imágenes se capturan de frente a la góndola, con resolución e iluminación suficientes. • Cada imagen del lote puede asociarse a una ubicación registrada (Pasillo / Góndola / Nivel) en la carpeta de ubicaciones del proyecto. • Existe conexión a Internet para el envío de alertas por correo electrónico. • Dependencia de librerías de terceros: Ultralytics (entrenado con v8.4.146), PyTorch, OpenCV, Albumentations, Roboflow SDK. • Se asume que el dataset público es suficientemente representativo de las góndolas reales hasta contar con imágenes propias.
10.6 Requisitos futuros	• Incorporar imágenes reales de salas Walmart etiquetadas para reentrenar el modelo. • Crear un set de prueba (test) independiente y ampliar el set de validación. • Evaluar variantes de mayor capacidad (YOLOv8s/m) si el hardware lo permite. • Evaluar un cálculo del % de stock ponderado por área de las cajas (ver 12.3). • Exportar el modelo a formatos optimizados (ONNX) para ejecución en equipos de sala. • Integración con los sistemas de inventario del cliente.

11. Pipeline de IA — Documentación técnica del notebook

Flujo implementado en Proyecto_CVW_modelo_v1.ipynb: Dataset Roboflow (bboxes) → Partición train/valid → Data augmentation (solo train) → Generación de data.yaml → Entrenamiento YOLOv8n → Validación → Inferencia de prueba → Exportación de best.pt.

11.1 Fuente de datos y partición

Parámetro	Valor	Detalle / celda del notebook
Origen	Roboflow Universe	Proyecto 'empty-spaces-in-a-supermarket-hanger-1upsp', versión 3, formato YOLOv8 (celda 3).
Clases (nc = 2)	0: missing · 1: product	Hueco / espacio vacío y producto visible.
Partición	85 % train / 15 % valid	La v3 solo trae 'train'; el split se crea antes de aumentar con semilla 42 para que la validación quede limpia (celda 4).
Imágenes	170 train originales · 29 valid	El set de validación contiene 4.449 instancias: 4.332 product y 117 missing.
Train final	680 imágenes	170 originales + 3 copias aumentadas por imagen (N = 3).

11.2 Aumento de datos (celdas 5, 6 y 12)

Parámetro	Valor
Librería	Albumentations
Alcance	Solo el set de entrenamiento; el set de validación queda sin aumentar.
Copias por imagen	N = 3 → 170 originales + 510 aumentadas = 680 imágenes de train
Transformaciones	Lumínicas / de degradación y geométricas, ajustando las bounding boxes.
Imágenes ilegibles	Se omiten sin detener el proceso (ver RNF-06).

11.3 Configuración del entrenamiento (celda 8)

Parámetro	Valor
Modelo base	yolov8n.pt (pre-entrenado; se transfirieron 319/355 capas) — 3,0 M parámetros, 8,1 GFLOPs
Épocas / paciencia	100 épocas máximo · early stopping con patience = 20
Tamaño de imagen / batch	640 px · 16
Optimizador	optimizer = auto → Ultralytics seleccionó AdamW (lr = 0,001667, momentum = 0,9)
Semilla	42 (reproducibilidad)
Entorno	Google Colab · GPU NVIDIA Tesla T4 (15 GB) · Ultralytics 8.4.146 · PyTorch 2.11
Resultado del entrenamiento	Detenido en la época 91; mejor modelo en la época 71 (best.pt); duración 0,93 h

11.4 Resultados de validación

Clase	Imágenes	Instancias	Precisión	Recall	mAP50	mAP50-95
Global (all)	29	4.449	78,5 %	67,5 %	72,8 %	38,8 %
missing	27	117	71,9 %	53,8 %	59,9 %	28,9 %
product	29	4.332	85,1 %	81,2 %	85,7 %	48,7 %

Etapa de inferencia (por imagen, GPU T4)	Tiempo
Preprocesamiento	0,2 ms
Inferencia	6,4 ms
Postprocesamiento (NMS)	2,1 ms
Total	**8,7 ms** (requisito RNF-01: < 5 s)

11.5 Datos generados

Artefacto	Descripción
runs/detect/train-6/weights/best.pt	Pesos del mejor modelo (6,3 MB). Se descarga desde Colab (celda 15) para la inferencia.
runs/detect/train-6/weights/last.pt	Pesos de la última época entrenada.
final_dataset/data.yaml	Rutas de train/valid, nc = 2 y nombres de clases (celda 7).
runs/detect/predict/	Imágenes anotadas con las detecciones de la prueba rápida (celdas 9–10).

12. Lógica de negocio — Semáforo operativo

12.1 Cálculo del % de stock visible

El % de stock visible se calcula por imagen a partir de los conteos de detecciones de ambas clases que entrega el modelo:

% stock visible = detecciones 'product' ÷ (detecciones 'product' + detecciones 'missing') × 100

Es un cálculo liviano, por conteo, que no agrega procesos que afecten el funcionamiento del sistema.

12.2 Umbrales del semáforo

Estado	Rango de stock visible	Alerta
Verde	100 % – 41 %	No
Amarillo	40 % – 21 %	No
Rojo	20 % – 0 %	Sí: correo electrónico con la ubicación (Pasillo / Góndola / Nivel)

Umbrales definidos en el Informe Final del Proyecto APT. El objetivo general contempla que el criterio de clasificación se designe según el recinto en que se implemente.

12.3 Regla de alerta y consideraciones

Cuando una imagen queda en estado Rojo (20 % o menos de stock visible), el sistema obtiene su ubicación desde la carpeta de ubicaciones del proyecto y despacha la alerta por correo electrónico.

El cálculo se basa en el conteo de cajas y no pondera su tamaño; un cálculo ponderado por área queda como requisito futuro (ver 10.6).

13. Requisitos Específicos

13.1 Requisitos comunes de las interfaces

13.1.1 Interfaces de usuario	• Correo electrónico: mensaje de alerta con ubicación, % de stock y estado (imagen anotada como adjunto deseable). • Demo web interactiva (HTML/CSS/JS) publicada en GitHub Pages para visualizar detecciones y semáforo. • Consola / scripts en VS Code para el administrador técnico (carga de lotes y ejecución).
13.1.2 Interfaces de hardware	• Entrenamiento: Google Colab con GPU NVIDIA Tesla T4. • Inferencia: laptop de gama media (CPU o GPU). • Captura: cualquier dispositivo que genere imágenes de la góndola (no se usa la red de CCTV).
13.1.3 Interfaces de software	• Python 3 · Ultralytics YOLOv8 · PyTorch · OpenCV · Albumentations · Roboflow SDK. • Carpeta de ubicaciones dentro del proyecto (pasillo, góndola, nivel), sin base de datos externa. • Servicio de correo electrónico para las notificaciones y API del sistema.
13.1.4 Interfaces de comunicación	• Conexión saliente hacia el servicio de correo electrónico para el envío de alertas. • HTTPS hacia la API de Roboflow solo en la etapa de preparación del dataset. • Sin conexiones RTSP ni acceso a la red corporativa de Walmart.

13.2 Requisitos funcionales

ID	Nombre	Descripción	Prioridad	Implementación
RF-01	Descargar dataset	Descargar el dataset de Roboflow en formato YOLOv8.	Alta	Sí — celda 3
RF-02	Particionar dataset	Generar un split de validación del 15 % con semilla 42, antes de aumentar.	Alta	Sí — celda 4
RF-03	Aumentar datos	Generar 3 copias aumentadas por imagen de train (degradación + geometría) ajustando las bounding boxes; valid sin aumentar.	Alta	Sí — celdas 5, 6 y 12
RF-04	Generar configuración	Crear data.yaml con rutas, nc = 2 y nombres de clases.	Alta	Sí — celda 7
RF-05	Entrenar modelo	Entrenar YOLOv8n (100 épocas, 640 px, batch 16, patience 20) y guardar best.pt.	Alta	Sí — celda 8
RF-06	Validar modelo	Reportar Precisión, Recall, mAP50 y mAP50-95 globales y por clase.	Alta	Sí — celda 8

ID	Nombre	Descripción	Prioridad	Implementación
RF-07	Inferencia por lotes	Procesar un conjunto de imágenes y entregar por cada una las detecciones (clase, confianza, bbox) y la imagen anotada.	Alta	Parcial — celdas 9–10 (una imagen)
RF-08	Calcular % de stock	Calcular el % de stock visible por imagen según la fórmula de la sección 12.1.	Alta	Fuera del notebook — Sprint 3
RF-09	Clasificar semáforo	Asignar Verde (100–41 %), Amarillo (40–21 %) o Rojo (20–0 %), según la sección 12.2.	Alta	Fuera del notebook — Sprint 3
RF-10	Consultar ubicación	Obtener Pasillo / Góndola / Nivel de la imagen desde la carpeta de ubicaciones del proyecto.	Alta	Fuera del notebook — Sprint 3
RF-11	Enviar alerta	Ante estado Rojo, enviar un correo electrónico con la ubicación y el % de stock. Adjuntar la imagen anotada es deseable.	Alta	Fuera del notebook — Sprint 4
RF-12	Exportar modelo	Descargar best.pt para su uso fuera de Colab.	Media	Sí — celda 15
RF-13	Visualizar resultados	Mostrar detecciones y semáforo en la demo web.	Media	Demo web (GitHub Pages)
RF-14	Disponer de API	Contar con una API propia del sistema, creada en el Sprint 4. Sus endpoints y tecnología se especificarán cuando el integrante responsable entregue su definición.	Media	Sprint 4 — detalle pendiente

13.3 Requisitos no funcionales

ID	Categoría	Descripción
RNF-01	Rendimiento	El tiempo de inferencia por imagen debe ser menor a 5 s. Medido: 8,7 ms en GPU T4; se debe validar también en el equipo de ejecución.
RNF-02	Calidad del modelo	Toda nueva versión del modelo debe igualar o superar la línea base v1 (mAP50 global 72,8 %; mAP50 missing 59,9 %).
RNF-03	Seguridad	El sistema no accede a cámaras, CCTV ni a la red corporativa; solo procesa imágenes entregadas por lotes.
RNF-04	Privacidad	Las imágenes se usan solo para analizar góndolas; no se almacenan ni procesan datos personales de clientes.
RNF-05	Gestión de credenciales	Las claves y credenciales (API de Roboflow, cuenta de correo de alertas) se guardan en variables de entorno o secretos, nunca en el código.
RNF-06	Fiabilidad	Una imagen que no se puede leer se omite sin detener el lote (comportamiento ya implementado en la aumentación).
RNF-07	Reproducibilidad	Semilla fija (42) en la partición y el entrenamiento; validación sobre imágenes limpias no aumentadas.
RNF-08	Disponibilidad	El sistema opera bajo demanda al recibir cada lote; no requiere funcionamiento 24/7.
RNF-09	Mantenibilidad	Parámetros centralizados (dataset, versión, N de aumentos, modelo base) y código modular en VS Code; el modelo base se puede cambiar (n/s/m/l) sin modificar el pipeline.
RNF-10	Portabilidad	Modelo liviano (best.pt de 6,3 MB) ejecutable en CPU o GPU sobre cualquier sistema con Python.

14. Arquitectura — Modelo de Vistas 4+1 (Kruchten)

Vista	Contenido en CVW
Escenarios (+1)	Caso principal: el administrador carga un lote → el sistema detecta, calcula el semáforo → si hay Rojo, el reponedor recibe la alerta por correo electrónico.
Lógica	Componentes: Ingesta de lotes, Detector YOLOv8n, Calculadora de semáforo, Carpeta de ubicaciones, Notificador (correo electrónico), API, Visualizador web.
Procesos	Flujo secuencial por lote: Lectura de imagen → Inferencia → Conteo por clase → % Stock → Clasificación → Consulta de ubicación en la carpeta → Alerta por correo.
Desarrollo	Notebook de entrenamiento en Colab; proyecto modular en VS Code (módulos de visión, lógica, ubicaciones, alertas y API); demo web en GitHub Pages.
Física	Google Colab (GPU T4) para entrenar; laptop para inferencia y carpeta de ubicaciones; servicio de correo electrónico para la entrega de alertas; GitHub Pages para la demo.